

Embeddings for Search

Meaning as distance · video 2:14

This lesson answers one question: how can a computer find text by **meaning**, not by matching words? An **embedding model** turns a whole text into one vector (here 384 numbers), so that similar meanings point in similar directions. A question becomes a vector too, and **cosine similarity** ranks the documents. This is the search step behind RAG (next episode).

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. One vector per text	▶ 0:08-0:47	3	☐
2. A map of meaning	▶ 0:47-1:03	2	☐
3. Searching with cosine similarity	▶ 1:03-1:38	3	☐
4. At scale, and the limits	▶ 1:38-2:11	3	☐

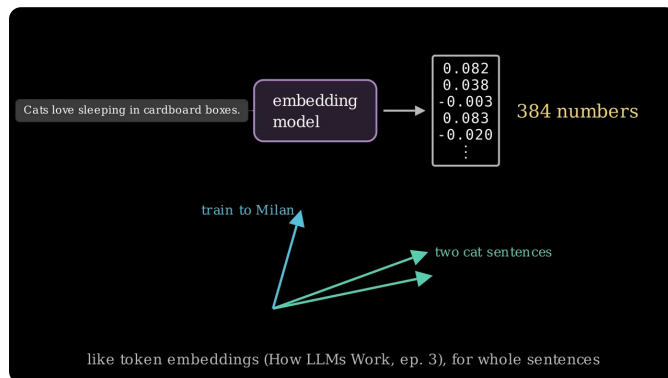
Before you start: How LLMs Work episode 3 (embeddings) and Foundations F01-F02 (vectors, the dot product and cosine similarity). Code: `code/p04_embedding_search` (downloads a 90 MB model, runs on a CPU).

CONCEPT 1 OF 4

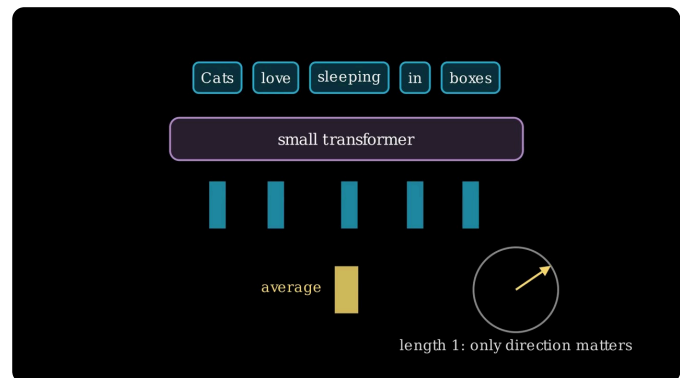
One vector per text

▶ Video 0:08-0:47

Keyword search matches **letters**, not meaning: searching your notes for “kitten” finds nothing, although you wrote about your cat twice. An **embedding model** reads a whole piece of text and returns a single **vector**; for all-MiniLM-L6-v2, **384 numbers**. Texts with similar meanings get vectors that point in similar directions: the same idea as token embeddings (How LLMs Work, episode 3), for whole sentences.



▶ 0:34 An embedding model turns a sentence into 384 numbers (the first five are real). Similar meanings point in similar directions.



▶ 0:46 Inside: a small transformer, the token vectors averaged into one, then scaled to length 1.

Under the hood it is a small **transformer**: each token comes out as a vector, the vectors are **averaged** into one, and the result is **scaled to length 1**, so only its direction matters.

KEY IDEA

An embedding is a text's **meaning as a direction** in a high-dimensional space.

Check 1: One vector per text

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

What does all-MiniLM-L6-v2 return for the sentence “Cats love sleeping in cardboard boxes.”?

- A. One vector of 384 numbers
- B. One vector per word
- C. A list of keywords
- D. A probability for the next token

1.2 TRUE OR FALSE

“After scaling to length 1, two sentence vectors can only differ in direction.”

True ☐ False ☐

1.3 SHORT ANSWER

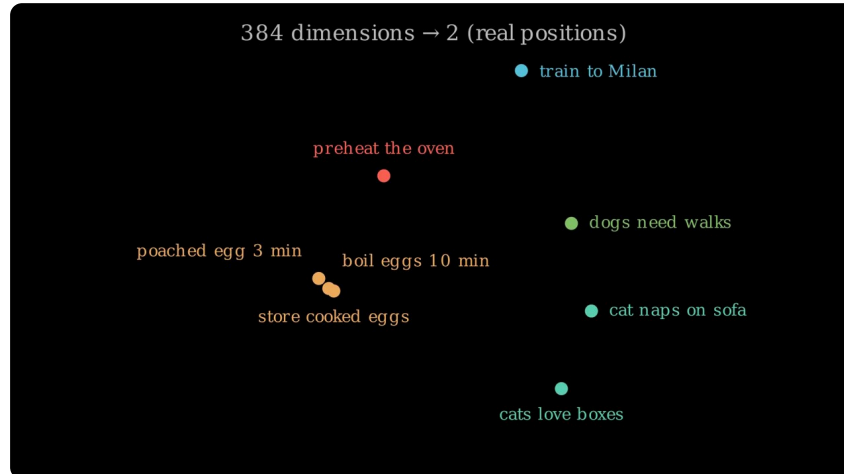
Why does keyword search find nothing for “kitten” in notes about a cat?

CONCEPT 2 OF 4

A map of meaning

▶ Video 0:47–1:03

To look at the vectors we can squash 384 dimensions down to two (with PCA). These are the real positions of our eight sentences: the three about **eggs** land together, the two about **cats** sit side by side, and the **train to Milan** is off on its own.



▶ 1:02 The 8 sentences squashed from 384 dimensions to 2 (real PCA positions): eggs together, cats together, the train apart.

KEY IDEA

Similar topics form **clusters**. A 2-D picture loses most of the information, but it shows the idea.

Check 2: A map of meaning

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 TRUE OR FALSE

“In the 2-D map, the two cat sentences are close because they share many words.”

True ☐ False ☐

2.2 SHORT ANSWER

Why can the 2-D map be misleading?

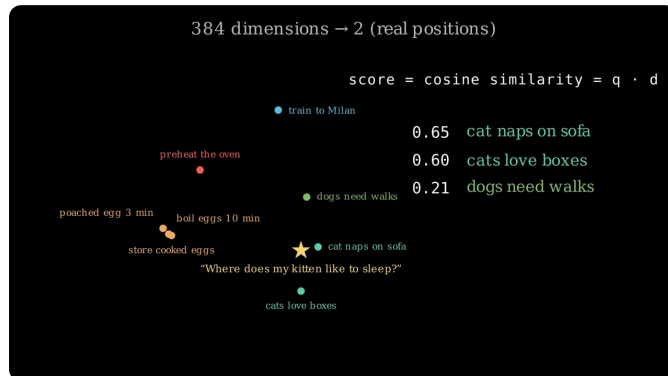
.....

CONCEPT 3 OF 4

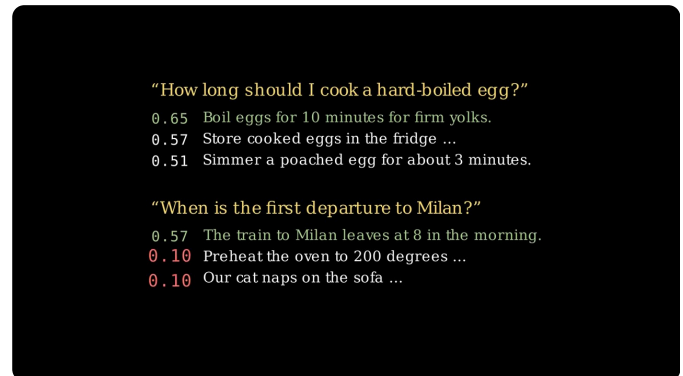
Searching with cosine similarity

▶ Video 1:03-1:38

A question becomes a vector too. “Where does my kitten like to sleep?” lands right next to the cats. To rank the documents we take the **cosine similarity**; for length-1 vectors it is just a **dot product**. Cat on the sofa: **0.65**. Cats in boxes: **0.60**. Dogs: only **0.21**. And not one word in common.



▶ 1:24 The kitten question lands next to the cats; the top two scores are the cat sentences, with zero words in common.



▶ 1:37 Two more real queries: the right document wins each time.

Ask how long to cook a hard-boiled egg, and the best match is “Boil eggs for 10 minutes” (0.65). Ask about the first departure to Milan, and the train wins easily (0.57); everything else scores close to zero (0.10).

KEY IDEA

Search = embed the question, take its dot product with every document vector, return the highest.

Check 3: Searching with cosine similarity

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Two length-1 vectors in 3-D are $a = (0.6, 0.8, 0)$ and $b = (0.8, 0.6, 0)$. What is their cosine similarity?

3.2 MULTIPLE CHOICE

Cosine similarity of 0 between two length-1 vectors means:

- A. They are identical
- B. They point at right angles: unrelated directions
- C. They point in opposite directions
- D. One of them is zero

3.3 SHORT ANSWER

The Milan question scores 0.57 with the train sentence and 0.10 with the next best. What does that big gap tell you?

CONCEPT 4 OF 4

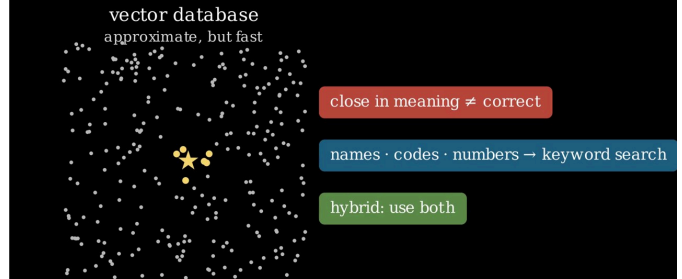
At scale, and the limits

▶ Video 1:38-2:11

In code: embed your documents **once** and store the vectors. When a question comes in, embed it, take its dot product with every stored vector, and return the top few.

```
1 doc_vectors = embed(documents)      # once, then store them
2
3 def search(question, k=3):
4     q = embed([question])[0]
5     scores = doc_vectors @ q          # cosine similarity
6     return topk(scores, k)
```

▶ 1:48 The whole search in a few lines: embed once, then a dot product per document.



▶ 2:05 Vector databases find nearest neighbours fast; keyword search still wins for exact names, codes and numbers.

With millions of documents, a **vector database** finds the nearest neighbours fast by searching **approximately**. But close in meaning is not the same as **correct**, and for exact things like names, codes or numbers, plain **keyword search** still wins. Many systems combine both (**hybrid search**).

KEY IDEA

Embeddings find what is **about** the same thing; keywords find what **says** the same thing. Good search often needs both.

Check 4: At scale, and the limits

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

A user searches a parts catalogue for “part number 7741-B”. Which search is most reliable?

- A. Embedding search only
- B. Keyword (exact) search
- C. Sampling at high temperature
- D. A 2-D PCA map

4.2 TRUE OR FALSE

“The top result of an embedding search is always correct.”

True ☐ False ☐

4.3 HANDS-ON CODE

Try it yourself. Add these lines at the end of `search.py` and run it. (a) Which document is the top match for each question, with what score? (b) The top egg score is lower than for the hard-boiled question (0.65). Why might that be? (c) Compute the similarity of “I love my cat” and “I love my car” and explain the result.

```
search("Can I still eat a boiled egg I made five days ago?")
search("My puppy is full of energy")
a, b = embed(["I love my cat", "I love my car"])
print(round(float(a @ b), 2))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. One vector per text · rewatch ▶ 0:08-0:47

1.1 A. Token vectors are averaged into a single sentence vector.

1.2 True. All vectors have the same length, so only the direction carries meaning.

1.3 The notes share no word with the query; keyword search compares letters, not meaning. “kitten”, “cat” and “cats” are different strings.

2. A map of meaning · rewatch ▶ 0:47-1:03

2.1 False. “Cats love sleeping in cardboard boxes” and “Our cat naps on the sofa all afternoon” share almost no words; they are close because of meaning.

2.2 Squashing 384 dimensions into 2 throws most of the information away; points close in 2-D are not always close in 384-D. Search always uses the full vectors, never the 2-D picture.

3. Searching with cosine similarity · rewatch ▶ 1:03-1:38

3.1 0.96. $0.6 \times 0.8 + 0.8 \times 0.6 + 0 \times 0 = 0.48 + 0.48 = 0.96$.

3.2 B. The dot product of perpendicular vectors is 0 (Foundations F02).

3.3 The search is confident: one document clearly matches, and the rest are unrelated. When the top scores are close together, several documents are similarly relevant.

4. At scale, and the limits · rewatch ▶ 1:38-2:11

4.1 B. Exact codes have no “meaning” for an embedding model; matching the string is what you want.

4.2 False. It is the closest in meaning among your documents, which may still be wrong or irrelevant if no document answers the question.

4.3 (a) “Store cooked eggs in the fridge for up to a week.” (0.56); “Dogs need a walk at least twice a day.” (0.47). (b) The question is about safety and time, which no document states directly, so even the best match is only partly related. (c) 0.60: the sentences share their structure and most words, so they score high although a cat and a car mean very different things. Similarity mixes topic, wording and structure; it is a useful signal, not a guarantee of meaning.